MINI PROJECT REPORT

On

AI DOCUMENT SUMMARIZER

An Intelligent Extractive, Abstractive and Hybrid Summarization Platform for Long-Form Documents

SUBMITTED BY

Vinay Guleria

UID - 25MCI10004

Anish Sharma

UID - 25MCI10065

Rishav Thakur

UID - 25MCI10008

UNDER THE GUIDANCE OF

[Guide Name]

[Designation], University Institute of Computing (UIC)

in partial fulfilment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS (AI&ML)

Chandigarh University

Certificate

This is to certify that **Vinay Guleria** (UID - 25MCI10004), **Anish Sharma** (UID - 25MCI10065) and **Rishav Thakur** (UID - 25MCI10008), students of Master of Computer Applications (MCA) — Artificial Intelligence & Machine Learning, have successfully completed the Minor Project titled **"AI Document Summarizer"** under the esteemed guidance of **[Guide Name]**, University Institute of Computing (UIC), Chandigarh University.

The project has been undertaken as part of the academic curriculum and is submitted in partial fulfilment of the requirements for the MCA (AI & ML) programme. The work presented in this project reflects the students' independent research, dedication and technical competence, demonstrating the ability to apply theoretical knowledge to address real-world problems effectively.

The project, **"AI Document Summarizer"**, is a full-stack web application that ingests documents in PDF, DOCX and TXT formats and produces summaries using three complementary strategies — extractive, abstractive and hybrid — with Optical Character Recognition (OCR) for scanned pages and reference-based evaluation using ROUGE and BLEU. It demonstrates key concepts of Natural Language Processing, Machine Learning and modern web application development, using a Python (FastAPI) backend with pretrained transformer models and a React front end.

We hereby certify that this project is original work carried out by the students and has not been submitted elsewhere for the award of any other degree, diploma or certificate.

Project Guide:

[Guide Name]

[Designation]

University Institute of Computing

Chandigarh University

Acknowledgement

We would like to express our sincere gratitude to Chandigarh University and the University Institute of Computing (UIC) for providing us with the opportunity to undertake the Minor Project titled **"AI Document Summarizer"**.

We extend our heartfelt appreciation to our esteemed guide, **[Guide Name]**, for the invaluable guidance, continuous support and insightful feedback provided throughout the development of this project. The expertise and mentorship were instrumental in shaping our understanding and in the successful completion of this work.

We are grateful to the Department of Computer Applications and the faculty members for providing the laboratory facilities, software environment and academic support needed to complete this work. We also thank our friends and peers for their constant encouragement, constructive suggestions and valuable discussions, which helped us refine our approach and improve the project outcomes.

Finally, we would like to express our profound gratitude to our families for their unwavering support, patience and motivation throughout this journey. The open-source community deserves recognition as well — this project is built almost entirely on open-source libraries and openly available pretrained machine learning models, without which the work would not have been possible within the scope of a minor project.

This project has been an enriching learning experience that strengthened our skills in Natural Language Processing, transformer-based machine learning and full-stack web development.

Contents

Section	Page Number
Abstract	5
1. Introduction	6
2. Tools and Technologies Used	9
3. Flowchart	11
4. Source Code	13-45
5. Results / Output Screenshot	46
6. Conclusion	49
7. References	50

Abstract

Long-form digital documents — research papers, project specifications, legal contracts, government reports and scanned records — grow far faster than any person can read them. Automatic text summarization addresses this by producing a short, readable form that preserves a document's essential information. In this context the **AI Document Summarizer** has been developed as a full-stack solution that combines machine learning and modern web technologies to deliver reliable, real-time summarization.

The platform ingests documents in **PDF, DOCX and TXT** formats and produces summaries using **three complementary strategies**: *extractive*, *abstractive* and *hybrid*. By replacing slow and subjective manual summarization with an automated system, it ensures faster processing, improved consistency and reduced human effort, while exposing the trade-off between factual fidelity and linguistic fluency.

Technically, the project is designed as a processing **pipeline** rather than a single model call. Every document passes through the same five stages — **ingestion, cleaning, chunking, summarization and evaluation** — and only the summarization stage differs by user choice. The backend is powered by **Python** with the **FastAPI** framework, which enables lightweight API development, automatic validation and seamless integration with machine learning models.

The **extractive** strategy embeds every sentence using the `all-MiniLM-L6-v2` sentence-transformer model, builds a sentence-similarity graph and runs **TextRank** (PageRank) over it to select the most representative sentences verbatim. Because nothing is ever rewritten, this mode cannot hallucinate. The **abstractive** strategy uses the `facebook/bart-large-cnn` sequence-to-sequence transformer to generate new, fluent sentences, and applies a **map-reduce** scheme with sentence-aligned overlapping chunks so that documents exceeding the model's 1024-token limit can still be summarized. The **hybrid** strategy first filters the document to its highest-ranked 50% of sentences using TextRank and then feeds only those sentences to BART, constraining generation to factually central content and reducing the risk of hallucination. Summaries can optionally be scored against a human-written reference using **ROUGE-1, ROUGE-2, ROUGE-L and BLEU** metrics.

On the front end, technologies such as **React, Vite and Tailwind CSS** are used to create an interactive and responsive single-page interface for upload, strategy selection, summary display and evaluation. Machine learning models are loaded lazily as process-level singletons, and blocking inference is dispatched to worker threads so that the HTTP event loop remains responsive. The system is validated by an automated test suite of **89 tests** spanning text extraction, cleaning, sentence segmentation, chunking, OCR orchestration and the API layer.

The project demonstrates the practical integration of artificial intelligence with modern web development, showing how contemporary technologies can build scalable, efficient and user-friendly systems that improve how long documents are read and understood.

Keywords: text summarization, extractive summarization, abstractive summarization, TextRank, BART, OCR, ROUGE, BLEU, FastAPI, React.

1. Introduction

1.1 Overview

Reading is no longer the bottleneck it once was for *access* to information — but it is now the bottleneck for *processing* information. A single research paper runs to ten or more pages, a project specification document to fifty, and a government or legal report to hundreds. A reader who needs to know whether a document is relevant, or what its central claims are, cannot afford to read every word.

AI Document Summarizer is a web application that accepts such documents and returns a short, information-dense summary. It supports the three file formats most commonly encountered in academic and professional work — **PDF**, **DOCX** and **TXT** — and offers three summarization strategies with different trade-offs between factual fidelity and linguistic fluency.

The application is deliberately structured as a pipeline rather than a single model call. Every document passes through the same five stages: **ingestion, cleaning, chunking, summarization and evaluation**. The summarization stage is the only one that differs by user choice; the rest of the pipeline is shared. This structure is what allows a scanned, badly formatted PDF and a clean TXT file to be handled by the same downstream code.

1.2 Motivation and Problem Statement

Manual summarization is slow, subjective and expensive. A professional summarising a 40-page document may spend an hour on it, and the same document summarised by a colleague will produce a different result. Machine summarization is fast, consistent and cheap and — while it will never replace careful human reading — it is well suited to triage, indexing and first-pass comprehension.

The problem this project addresses can be stated precisely:

> **Given an arbitrary long-form document in PDF, DOCX or TXT format — possibly scanned, possibly poorly formatted — produce a concise summary of it, while allowing the user to choose between guaranteed factual fidelity and fluent paraphrasing, and optionally measuring the result against a human-written reference.**

Three sub-problems follow from this statement and shaped the design of the system:

1. **Extraction.** Documents are not always stored as clean text. PDFs are page-layout grids, scanned PDFs contain no text at all, and DOCX files split content across paragraphs and tables. A single summarizer cannot begin until these differences are normalised away.
2. **Model limits.** The abstractive model used — BART — reads at most 1024 tokens. A twenty-page document is far longer. The document must therefore be split, summarized piecewise and the results recombined, without cutting sentences in half in the process.
3. **Hallucination.** Abstractive models generate new sentences, and generated sentences can assert things the source document never said. For academic, legal or medical documents this is a serious

failure mode. The system must offer a strategy that cannot hallucinate, and a middle path that constrains generation to factually central content.

1.3 Objectives

The objectives of the project are:

1. To build a document ingestion subsystem that extracts text from **PDF, DOCX and TXT** files, including an **OCR fallback** for scanned PDF pages that carry no text layer.
2. To implement a **text cleaning** stage that converts extracted page layout back into flowing prose by removing page numbers and repeated headers/footers, re-joining words broken across line breaks and normalising characters.
3. To implement **sentence-aware, overlapping chunking** so that documents longer than the model's context window can be processed without fragmenting sentences.
4. To implement **extractive summarization** using sentence embeddings and the **TextRank** graph-ranking algorithm, with a guarantee that every output sentence is quoted verbatim from the source.
5. To implement **abstractive summarization** using the **BART** sequence-to-sequence transformer with a **map-reduce** scheme for long documents.
6. To implement **hybrid summarization** that pre-filters with TextRank before abstractive condensation, reducing hallucination risk while retaining fluency.
7. To implement **automatic evaluation** of a generated summary against a human reference using **ROUGE-1, ROUGE-2, ROUGE-L and BLEU**.
8. To build a responsive **single-page web interface** for upload, strategy selection, summarization and evaluation.
9. To validate the system with an **automated test suite** covering extraction, cleaning, segmentation, chunking, OCR orchestration and the HTTP API.

1.4 Scope of the Project

Included in scope:

- Document formats: PDF (digital and scanned), DOCX (paragraphs *and* tables), TXT (with encoding detection).
- Three summarization strategies: extractive, abstractive and hybrid.
- Three summary length settings: short, medium and long.
- OCR for English-language scanned pages, using Tesseract.
- Reference-based evaluation with ROUGE and BLEU.
- A local development deployment: FastAPI backend and React front end running on the same machine.

Excluded from scope:

- Languages other than English (both OCR and the pretrained models are configured for English).

- Multi-document or query-focused summarization.
- User accounts, authentication or a persistent summary history.
- Containerised or cloud deployment (identified as a later phase).
- GPU inference (the system runs on CPU, with configurable device support).

1.5 Organisation of the Report

This report follows the standard minor-project format. **Section 1** introduces the problem, the motivation and the objectives of the project. **Section 2** lists the tools and technologies used and the role each one plays. **Section 3** presents the flowcharts of the system. **Section 4** contains the source code of the core modules. **Section 5** shows the results and the output screens of the application. **Section 6** concludes the report, and **Section 7** lists the references.

2. Tools and Technologies Used

The AI Document Summarizer integrates modern web frameworks and machine learning libraries to deliver a fast, responsive and reliable summarization platform.

2.1 Programming Languages & Core

- **Python 3.11** — backend development, document processing and machine-learning inference.
- **HTML5** — structured semantic markup for the single-page interface.
- **CSS3** — styling through Tailwind CSS utility classes and a small custom theme.
- **JavaScript (ES6+)** — interactive behaviour in the React front end.

2.2 Frontend Framework Details

The user interface is built with a focus on clarity and responsiveness:

- **React 19 with Vite 8** — a single-page application whose view is driven by an explicit page state machine (`idle → uploading → uploaded → processing → completed | error`).
- **Tailwind CSS v4** — utility-first styling with a small design-token theme.
- **axios** — a single API module that centralises every HTTP call and normalises all error shapes into one readable message.
- **Responsive design** — a fluid layout that adapts from mobile to desktop.

2.3 Backend & AI Engine

- **FastAPI with Uvicorn** — the asynchronous web framework and ASGI server, providing automatic request validation, typed response models and OpenAPI documentation.
- **Pydantic v2** — request/response schemas and application settings.
- `sentence-transformers` **(**`all-MiniLM-L6-v2`**)** — 384-dimensional sentence embeddings used by the extractive strategy.
- `transformers` **(**`facebook/bart-large-cnn`**)** — the sequence-to-sequence model used by the abstractive and hybrid strategies.
- **NetworkX** — the PageRank implementation used for TextRank.
- **NumPy / scikit-learn** — similarity-matrix computation.

2.4 Document Processing, OCR & Evaluation

- **PyMuPDF (**`fitz`**)** — PDF text-layer extraction and page rasterisation.
- **python-docx** — DOCX paragraph and table extraction.
- **Tesseract via** `pytesseract` **and Pillow** — Optical Character Recognition for scanned PDF pages.
- **NLTK (Punkt)** — sentence segmentation, with a hand-written regex splitter as a fallback.
- `rouge-score` **and** `sacrebleu` — ROUGE-1/2/L and BLEU evaluation metrics.

2.5 Development Tools

- **Visual Studio Code** — the development environment.
- **Git and GitHub** — version control.
- **pytest and httpx** — the automated test suite (89 tests) and HTTP test client.

2.6 Hardware and Software Requirements

Hardware: a machine with a modern multi-core CPU, at least 8 GB of RAM (16 GB recommended, since the abstractive model occupies roughly 1.6 GB when loaded) and about 3 GB of free disk space for the pretrained model cache.

Software: Windows 10/11, macOS or Linux; Python 3.11 or later; Node.js 18 or later; Tesseract OCR (optional, required only for scanned PDFs); and a Chromium-based browser for the front end.

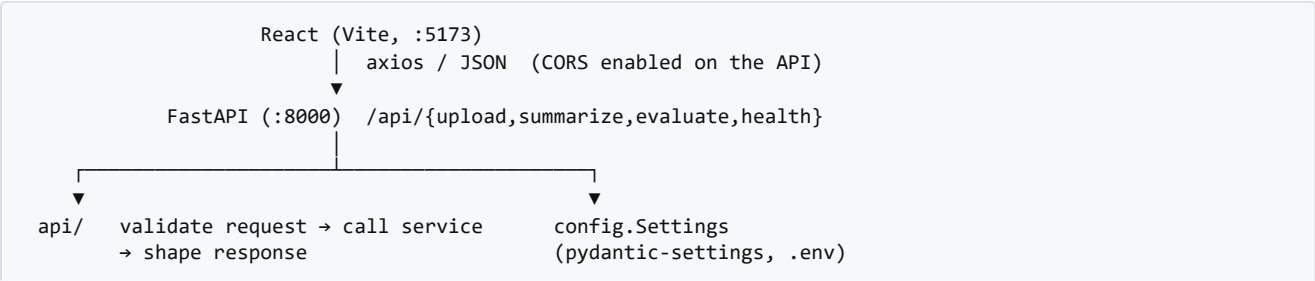
3. Flowchart

The following diagrams describe the system. Figure 3.1 shows the end-to-end processing pipeline that every document follows; Figure 3.2 shows how the modules depend on one another; and Figure 3.3 shows how the user's strategy choice selects a summarization path.

Figure 3.1 — End-to-end processing pipeline



Figure 3.2 — Module dependency graph



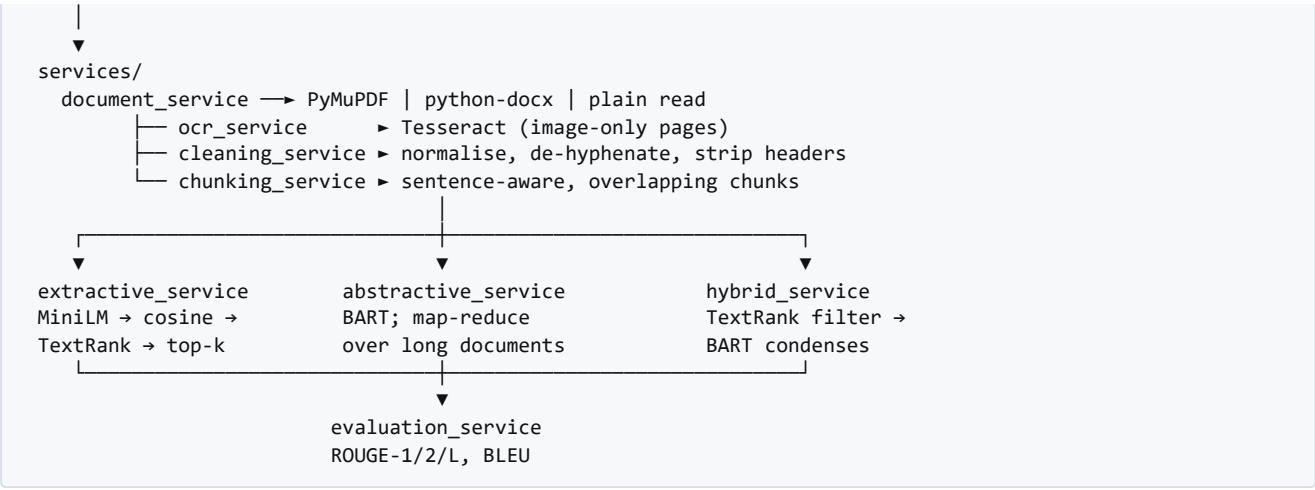
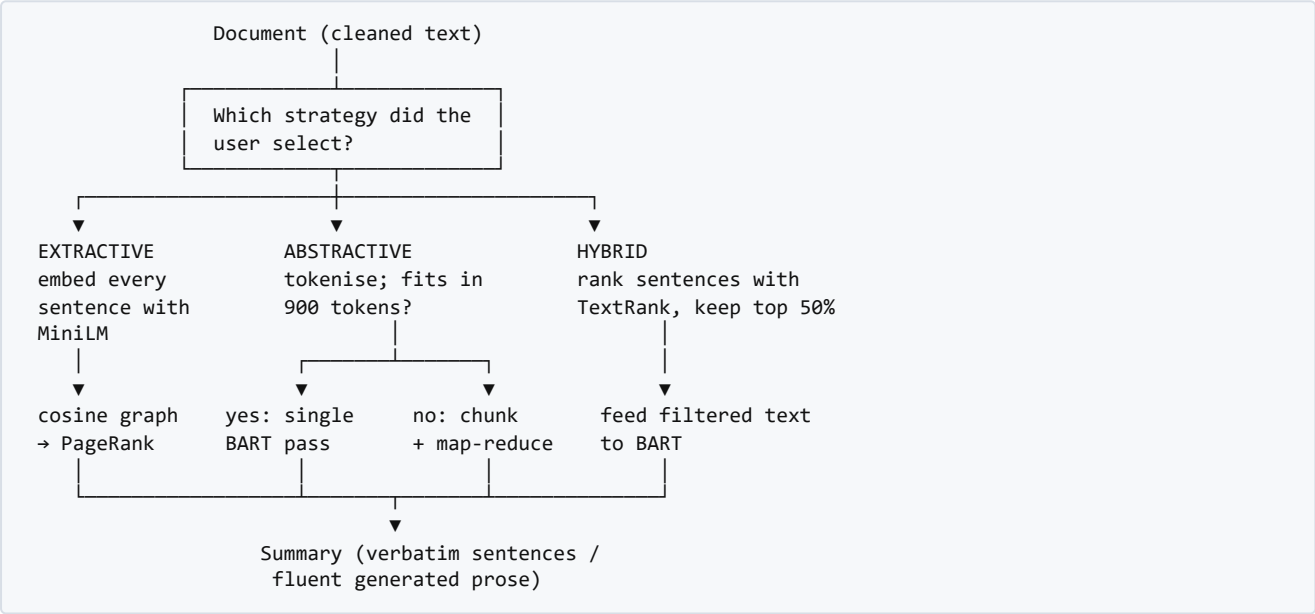


Figure 3.3 — Strategy selection



4. Source Code

This section presents the core source code of the project. The backend is a FastAPI application organised into an API layer (`app/api/`), a service layer (`app/services/`) and a shared configuration and schema layer; the front end is a React single-page application whose HTTP calls are centralised in a single API module. The complete code base, including the automated test suite, is provided in the project repository.

`backend/app/main.py`

```
"""
FastAPI application entry point.

Run from the `backend/` directory:

    .venv/Scripts/python.exe -m uvicorn app.main:app --reload --port 8000

Interactive API docs: http://localhost:8000/docs
"""

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from app.api import evaluate, health, summarize, upload
from app.config import settings

logging.basicConfig(
    level=logging.INFO, format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
)
logger = logging.getLogger(__name__)


@asynccontextmanager
async def lifespan(app: FastAPI):
    """Startup and shutdown tasks.

    Heavy models (Sentence-Transformers, BART) are loaded once here from Phase 5
    onwards, rather than on every request.
    """
    settings.ensure_directories()
    logger.info("Starting %s v%s", settings.APP_NAME, settings.APP_VERSION)
    logger.info("Uploads: %s", settings.upload_dir)
    yield
    logger.info("Shutting down")


app = FastAPI(
    title=settings.APP_NAME,
    version=settings.APP_VERSION,
    description=(
        "Extractive, abstractive and hybrid summarization for long-form "
        "documents (PDF / DOCX / TXT), with ROUGE and BLEU evaluation."
    ),
    lifespan=lifespan,
)

# The React dev server (port 5173) and the API (port 8000) are different
# origins, so the browser blocks requests unless the API allows them.
app.add_middleware(
    CORSMiddleware,
```

```
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


@app.exception_handler(RequestValidationError)
async def validation_error_handler(request: Request, exc: RequestValidationError):
    """Collapse FastAPI's verbose 422 body into one readable message.

    FastAPI normally returns a list of error objects; the front-end only ever
    has to render a single string.
    """
    problems = [
        f"{' -> '.join(str(p) for p in err.get('loc', ()) if p != 'body')}: {err.get('msg')}"
        for err in exc.errors()
    ]
    return JSONResponse(
        status_code=422, content={"detail": "; ".join(problems) or "Invalid request."}
    )


@app.exception_handler(Exception)
async def unexpected_error_handler(request: Request, exc: Exception):
    """Last resort: never leak a stack trace to the client.

    The traceback still goes to the server log, so debugging is unaffected.
    """
    logger.exception("Unhandled error on %s %s", request.method, request.url.path)
    return JSONResponse(
        status_code=500,
        content={"detail": "Unexpected server error. See the backend log for details."},
    )


app.include_router(health.router, prefix="/api")
app.include_router(upload.router, prefix="/api")
app.include_router(summarize.router, prefix="/api")
app.include_router(evaluate.router, prefix="/api")


@app.get("/", tags=["System"], summary="Service banner")
async def root() -> dict:
    return {
        "service": settings.APP_NAME,
        "version": settings.APP_VERSION,
        "docs": "/docs",
        "health": "/api/health",
    }
```

backend/app/config.py

```
"""Application settings, loaded from environment variables or `.env`."""

from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict

# This file is <root>/backend/app/config.py
PROJECT_ROOT = Path(__file__).resolve().parents[2]


def _resolve(value: str) -> Path:
    """Anchor a relative path to the project root.

    Without this, `UPLOAD_DIR=uploads` would mean different directories
    depending on where uvicorn was started from.
```

```
    """
    path = Path(value)
    return path if path.is_absolute() else PROJECT_ROOT / path


class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=str(PROJECT_ROOT / ".env"),
        env_file_encoding="utf-8",
        extra="ignore",
    )

    # ---- Application ----
    APP_NAME: str = "AI Document Summarization API"
    APP_VERSION: str = "0.1.0"

    # ---- Server ----
    BACKEND_HOST: str = "0.0.0.0"
    BACKEND_PORT: int = 8000

    # Comma-separated, not a list: pydantic-settings would try to JSON-decode a
    # list field before any validator runs, and "http://a,http://b" is not JSON.
    CORS_ORIGINS: str = "http://localhost:5173,http://127.0.0.1:5173"

    # ---- Uploads ----
    MAX_FILE_SIZE_MB: int = 20
    ALLOWED_EXTENSIONS: str = ".pdf,.docx,.txt"
    UPLOAD_DIR: str = "uploads"
    OUTPUT_DIR: str = "outputs"

    # ---- OCR (Phase 3) ----
    OCR_LANGUAGE: str = "eng"
    OCR_DPI: int = 200
    # A PDF page yielding fewer characters than this is treated as an image.
    OCR_MIN_CHARS_PER_PAGE: int = 80
    TESSERACT_CMD: str | None = None

    # ---- Chunking (Phase 4) ----
    CHUNK_SIZE: int = 1000
    CHUNK_OVERLAP: int = 100

    # ---- Cleaning (Phase 4) ----
    CLEAN_STRIP_HEADERS: bool = True
    # A short line repeating at least this many times is treated as a running
    # header or footer and removed. Higher = more conservative.
    HEADER_MIN_REPEATS: int = 3
    # Only lines this short are ever considered headers; a repeated full
    # sentence is content, however often it recurs.
    HEADER_MAX_CHARS: int = 120

    # ---- Models (Phases 5-7) ----
    EXTRACTIVE_MODEL: str = "sentence-transformers/all-MiniLM-L6-v2"
    ABSTRACTIVE_MODEL: str = "facebook/bart-large-cnn"
    ABSTRACTIVE_DEVICE: str = "cpu"

    # ---- Derived values ----
    @property
    def cors_origins(self) -> list[str]:
        return [o.strip() for o in self.CORS_ORIGINS.split(",") if o.strip()]

    @property
    def allowed_extensions(self) -> set[str]:
        return {e.strip().lower() for e in self.ALLOWED_EXTENSIONS.split(",") if e.strip()}

    @property
    def max_file_size_bytes(self) -> int:
        return self.MAX_FILE_SIZE_MB * 1024 * 1024

    @property
    def upload_dir(self) -> Path:
        return _resolve(self.UPLOAD_DIR)
```

```
    @property
    def output_dir(self) -> Path:
        return _resolve(self.OUTPUT_DIR)

    def ensure_directories(self) -> None:
        self.upload_dir.mkdir(parents=True, exist_ok=True)
        self.output_dir.mkdir(parents=True, exist_ok=True)


settings = Settings()
```

backend/app/models/schemas.py

```
"""
Pydantic request/response models.

These define the API's public shape. FastAPI validates requests against them,
serialises responses through them (so only declared fields are ever sent), and
generates the OpenAPI docs at `/docs` from them.
"""

from typing import Literal

from pydantic import BaseModel, Field

# Declaring these once means a typo like mode="extractiv" is rejected at the
# boundary with HTTP 422 instead of failing deep inside a service.
SummarizationMode = Literal["extractive", "abstractive", "hybrid"]
SummaryLength = Literal["short", "medium", "long"]


class CleaningInfo(BaseModel):
    """What the cleaning step changed, so its effect is visible, not silent.

    Cleaning removes page numbers and running headers/footers and rejoins words
    broken across a line. Showing the counts lets the user confirm that the
    document was read correctly rather than silently mangled.
    """

    raw_character_count: int = Field(description="Characters before cleaning.")
    removed_line_count: int = Field(
        default=0, description="Page numbers and repeated header/footer lines removed."
    )
    hyphenation_joins: int = Field(
        default=0, description="Words rejoined across a line break."
    )


class UploadResponse(BaseModel):
    """Metadata for a successfully ingested document.

    Note what is absent: the extracted text. The document is stored server-side
    under `document_id` and referred to by that id from then on.
    """

    document_id: str = Field(description="Handle used to summarize this document.")
    filename: str = Field(description="Sanitised original filename, for display.")
    file_type: str = Field(description="pdf, docx or txt.")
    character_count: int = Field(description="Characters of cleaned text.")
    word_count: int = Field(description="Words of cleaned text.")
    chunk_count: int = Field(
        default=0,
        description="Chunks the document is split into for summarization, using "
        "the current CHUNK_SIZE / CHUNK_OVERLAP settings.",
    )
    page_count: int | None = Field(
        default=None,
```

```
        description="Page count for PDFs; null for DOCX/TXT, whose page breaks "
        "cannot be determined reliably.",
    )
    ocr_used: bool = Field(description="Whether OCR was needed to recover the text.")
    extraction_method: str = Field(description="How the text was obtained.")
    cleaning: CleaningInfo = Field(description="What cleaning removed or repaired.")
    preview: str = Field(default="", description="First ~280 characters of the text.")
    warnings: list[str] = Field(
        default_factory=list, description="Non-fatal issues, e.g. image-only pages."
    )


class SummarizeRequest(BaseModel):
    """Ask for a summary of a document that was already uploaded."""

    document_id: str = Field(description="Id returned by /api/upload.")
    mode: SummarizationMode = Field(
        default="extractive", description="Which strategy to use."
    )
    length: SummaryLength = Field(
        default="medium", description="Roughly how long the summary should be."
    )


class SummarizeResponse(BaseModel):
    document_id: str
    mode: SummarizationMode
    length: SummaryLength
    summary: str = Field(description="The generated summary text.")


class ErrorResponse(BaseModel):
    """The single error shape every failing endpoint returns."""

    detail: str = Field(description="Human-readable explanation for the user.")


class EvaluationMetrics(BaseModel):
    rouge_1: float = Field(description="ROUGE-1 F1-score (0.0 to 1.0).")
    rouge_2: float = Field(description="ROUGE-2 F1-score (0.0 to 1.0).")
    rouge_l: float = Field(description="ROUGE-L F1-score (0.0 to 1.0).")
    bleu: float = Field(description="BLEU score (0.0 to 100.0).")


class EvaluateRequest(BaseModel):
    """Ask for evaluation of a document summary against a reference text."""

    document_id: str = Field(description="Id returned by /api/upload.")
    reference: str = Field(description="Reference summary text to evaluate against.")
    mode: SummarizationMode = Field(
        default="extractive", description="Which summarization strategy to use."
    )
    length: SummaryLength = Field(
        default="medium", description="Roughly how long the summary should be."
    )


class EvaluateResponse(BaseModel):
    document_id: str
    mode: SummarizationMode
    length: SummaryLength
    summary: str = Field(description="The generated summary text.")
    reference: str = Field(description="The reference summary text.")
    metrics: EvaluationMetrics = Field(description="Calculated ROUGE and BLEU metrics.")
```

backend/app/api/health.py

```
"""System health and readiness endpoints."""

from __future__ import annotations

import platform
import sys
from datetime import datetime, timezone

from fastapi import APIRouter

from app.config import settings
from app.services import ocr_service

router = APIRouter(tags=["System"])


@router.get("/health", summary="Liveness / readiness probe")
async def health_check() -> dict:
    """Report that the API process is up and how it is configured.

    The front-end calls this on load so the UI can tell the user immediately
    whether the backend is reachable, instead of failing later with a confusing
    upload error.
    """
    return {
        "status": "ok",
        "service": settings.APP_NAME,
        "version": settings.APP_VERSION,
        "python_version": sys.version.split()[0],
        "platform": platform.system(),
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "ocr": _ocr_status(),
    }


def _ocr_status() -> dict:
    """Describe whether scanned-PDF support is usable.

    A health endpoint must not be the thing that fails, so any error while
    probing Tesseract (or importing the wrapper) is reported as data rather than
    raised. OCR is optional: the API is healthy without it, it just cannot read
    scanned documents.
    """
    try:
        available, reason = ocr_service.availability()
    except Exception as exc:  # pragma: no cover - defensive
        available, reason = False, f"OCR status could not be determined ({exc})"

    status = {"available": available, "language": settings.OCR_LANGUAGE}
    if reason:
        status["reason"] = reason
    return status
```

backend/app/api/upload.py

```
"""
POST /api/upload - ingest a PDF, DOCX or TXT document.

Flow:
    validate filename and extension
      -> save to a temporary file with a size limit
      -> extract text (PyMuPDF / python-docx / plain read, with OCR for scans)
      -> clean it into flowing prose
      -> store the cleaned text under a document id
      -> delete the temporary upload
```

```
"""

import logging

from fastapi import APIRouter, File, UploadFile
from fastapi.concurrency import run_in_threadpool

from app.config import settings
from app.models.schemas import ErrorResponse, UploadResponse
from app.services import document_service
from app.utils.file_utils import (
    get_file_type,
    new_document_id,
    remove_file,
    safe_filename,
    save_upload,
)
from app.utils.text_utils import make_preview

logger = logging.getLogger(__name__)

router = APIRouter(tags=["Documents"])


@router.post(
    "/upload",
    response_model=UploadResponse,
    responses={
        400: {"model": ErrorResponse, "description": "Invalid or empty file"},
        413: {"model": ErrorResponse, "description": "File too large"},
        422: {"model": ErrorResponse, "description": "File could not be parsed"},
    },
    summary="Upload a document and extract its text",
)
async def upload_document(file: UploadFile = File(...)) -> UploadResponse:
    filename = safe_filename(file.filename)
    file_type = get_file_type(filename)

    document_id = new_document_id()
    temp_path = settings.upload_dir / f"{document_id}.{file_type}"

    try:
        await save_upload(file, temp_path)

        # Extraction and cleaning are blocking CPU/IO work. Running them in a
        # worker thread keeps the event loop free so /api/health and other
        # requests stay responsive while a large document is processed.
        result, cleaned = await run_in_threadpool(
            document_service.extract_and_clean, temp_path, file_type
        )
    finally:
        # The original binary is never needed again: only its text is stored.
        remove_file(temp_path)

    metadata = document_service.build_metadata(document_id, filename, result, cleaned)
    document_service.save_document(document_id, cleaned.text, metadata)

    logger.info(
        "Uploaded %s (%s): %d words, %d chunk(s), %s page(s)",
        filename,
        file_type,
        metadata["word_count"],
        metadata["chunk_count"],
        metadata["page_count"],
    )

    return UploadResponse(
        **metadata,
        preview=make_preview(cleaned.text),
```

```
        warnings=result.warnings,
    )
```

backend/app/api/summarize.py

```
"""
POST /api/summarize - summarize a document that was already uploaded.

The document is referred to by the id from /api/upload, so switching mode or
length never re-uploads the file.
"""

import logging

from fastapi import APIRouter, HTTPException
from fastapi.concurrency import run_in_threadpool

from app.models.schemas import ErrorResponse, SummarizeRequest, SummarizeResponse
from app.services import (
    abstractive_service,
    document_service,
    extractive_service,
    hybrid_service,
)

logger = logging.getLogger(__name__)

router = APIRouter(tags=["Summarization"])

SUMMARIZERS = {
    "extractive": extractive_service.summarize,
    "abstractive": abstractive_service.summarize,
    "hybrid": hybrid_service.summarize,
}


@router.post(
    "/summarize",
    response_model=SummarizeResponse,
    responses={
        404: {"model": ErrorResponse, "description": "Unknown document id"},
        501: {"model": ErrorResponse, "description": "Mode not implemented yet"},
    },
    summary="Summarize an uploaded document",
)
async def summarize_document(request: SummarizeRequest) -> SummarizeResponse:
    summarizer = SUMMARIZERS.get(request.mode)
    if summarizer is None:
        raise HTTPException(
            status_code=501,
            detail=f"Summarization mode '{request.mode}' is not implemented yet.",
        )

    text, _metadata = document_service.load_document(request.document_id)

    # Model inference is blocking CPU work, so it runs in a worker thread.
    summary = await run_in_threadpool(summarizer, text, request.length)

    return SummarizeResponse(
        document_id=request.document_id,
        mode=request.mode,
        length=request.length,
        summary=summary,
    )
```

backend/app/api/evaluate.py

```
"""
POST /api/evaluate - generate a summary and evaluate it against a reference text.

The document is referred to by the id from /api/upload. The requested mode
summarizes the document, and the result is scored against the user-provided
reference summary using ROUGE-1/2/L and BLEU.
"""

import logging

from fastapi import APIRouter, HTTPException
from fastapi.concurrency import run_in_threadpool

from app.api.summarize import SUMMARIZERS
from app.models.schemas import ErrorResponse, EvaluateRequest, EvaluateResponse
from app.services import document_service, evaluation_service

logger = logging.getLogger(__name__)

router = APIRouter(tags=["Evaluation"])


@router.post(
    "/evaluate",
    response_model=EvaluateResponse,
    responses={
        404: {"model": ErrorResponse, "description": "Unknown document id"},
        501: {"model": ErrorResponse, "description": "Mode not implemented yet"},
    },
    summary="Evaluate summary against a reference text",
)
async def evaluate_document(request: EvaluateRequest) -> EvaluateResponse:
    summarizer = SUMMARIZERS.get(request.mode)
    if summarizer is None:
        raise HTTPException(
            status_code=501,
            detail=f"Summarization mode '{request.mode}' is not implemented yet.",
        )

    text, _metadata = document_service.load_document(request.document_id)

    # Summarization model inference and scoring run in worker thread pool
    summary = await run_in_threadpool(summarizer, text, request.length)
    scores = await run_in_threadpool(
        evaluation_service.evaluate, summary, request.reference
    )

    return EvaluateResponse(
        document_id=request.document_id,
        mode=request.mode,
        length=request.length,
        summary=summary,
        reference=request.reference,
        metrics=scores,
    )
```

backend/app/services/document_service.py

```
"""
Document extraction (PDF / DOCX / TXT), cleaning, and storage of the text.

Pipeline
--------
`extract_text` returns what the file literally contains. `extract_and_clean`
runs that through `cleaning_service` to turn a grid of extracted lines back into
```

```
flowing prose. The upload endpoint uses the latter; `extract_text` stays public
because it is the extraction step on its own, and is tested as such.

Storage
-------
`/api/upload` processes the document once and saves the result under a
`document_id`:

    uploads/<document_id>.txt     cleaned text, UTF-8
    uploads/<document_id>.json    metadata (filename, counts, method, ...)

`/api/summarize` then sends only that id, so changing mode or length never
re-uploads the file. The original upload is deleted after extraction, because
only the text is needed from that point on.
"""

import json
import logging
from dataclasses import dataclass, field
from pathlib import Path

import fitz  # PyMuPDF (the module is also importable as `pymupdf`; `fitz` is
# the long-standing name and still works, though newer releases warn about it)
from docx import Document as DocxDocument
from fastapi import HTTPException

from app.config import settings
from app.services import chunking_service, cleaning_service, ocr_service
from app.services.cleaning_service import CleanedText
from app.utils.text_utils import count_characters, count_words, has_text

logger = logging.getLogger(__name__)

# Below this many characters a document is treated as having no usable content.
# Kept small: it catches empty or garbage input, not short-but-real documents.
MIN_DOCUMENT_CHARS = 40

# Pages/paragraphs are joined with a blank line so structural boundaries survive.
# Sentence segmentation and chunking later treat blank lines as hard breaks.
BLOCK_SEPARATOR = "\n\n"


@dataclass
class ExtractionResult:
    """Raw output of an extraction pass."""

    text: str
    file_type: str
    extraction_method: str
    page_count: int | None = None
    ocr_used: bool = False
    warnings: list[str] = field(default_factory=list)


@dataclass
class OcrOutcome:
    """What happened when OCR was offered the image-only pages of a PDF.

    `attempted` distinguishes "OCR was not needed / not possible" from "OCR ran";
    `unavailable_reason` is set only when Tesseract could not be used at all, so
    the caller can tell the user how to install it.
    """

    attempted: bool
    recovered: list[int] = field(default_factory=list)
    failed: list[int] = field(default_factory=list)
    unavailable_reason: str | None = None


# ---------------------------------------------------------------------------- #
# Extraction
```

```
# ---------------------------------------------------------------------------- #
def extract_text(path: Path, file_type: str) -> ExtractionResult:
    """Extract text from a document, dispatching on its file type."""
    if file_type == "pdf":
        return _extract_pdf(path)
    if file_type == "docx":
        return _extract_docx(path)
    if file_type == "txt":
        return _extract_txt(path)
    raise HTTPException(status_code=400, detail=f"Cannot process '{file_type}' files.")


def extract_and_clean(path: Path, file_type: str) -> tuple[ExtractionResult, CleanedText]:
    """The ingestion pipeline: extract the raw text, then clean it.

    Extraction stays a separate function so it can be tested on its own and so
    the raw text remains available; this is the step the upload endpoint uses.
    """
    result = extract_text(path, file_type)
    return result, cleaning_service.clean_text(result.text)


def _extract_pdf(path: Path) -> ExtractionResult:
    """Read the text layer of each PDF page, falling back to OCR for images.

    PyMuPDF returns an empty string for a page that is really a picture, because
    there is no text layer to read. Any page below
    `settings.OCR_MIN_CHARS_PER_PAGE` characters is therefore treated as an image
    and handed to Tesseract; pages that already have text are never re-processed,
    so a normal digital PDF costs nothing extra.
    """
    try:
        document = fitz.open(path)
    except Exception as exc:
        raise HTTPException(
            status_code=422,
            detail=f"The PDF could not be opened. It may be corrupt. ({exc})",
        )

    try:
        if document.needs_pass:
            raise HTTPException(
                status_code=422,
                detail="This PDF is password protected. Please upload an unlocked copy.",
            )

        page_count = document.page_count
        if page_count == 0:
            raise HTTPException(status_code=400, detail="The PDF contains no pages.")

        page_texts = [page.get_text("text") or "" for page in document]

        # A real page of prose has hundreds of characters; a scanned page has
        # almost none. This threshold is what decides whether OCR runs at all.
        scanned_pages = [
            number
            for number, page_text in enumerate(page_texts, start=1)
            if len(page_text.strip()) < settings.OCR_MIN_CHARS_PER_PAGE
        ]

        # OCR must happen while the document is still open: it renders pages, and
        # PyMuPDF cannot render a closed document.
        outcome = _recover_scanned_pages(document, page_texts, scanned_pages)
    finally:
        # Always release the native handle, or the temp file cannot be deleted
        # on Windows.
        document.close()

    # Joined only now, so it includes whatever OCR recovered above.
    text = BLOCK_SEPARATOR.join(page_texts)
    warnings = _pdf_warnings(page_count, scanned_pages, outcome)
```

```
    if not has_text(text):
        raise HTTPException(status_code=422, detail=_no_text_detail(outcome))

    return ExtractionResult(
        text=text,
        file_type="pdf",
        extraction_method=_pdf_method(page_count - len(scanned_pages), outcome),
        page_count=page_count,
        ocr_used=bool(outcome.recovered),
        warnings=warnings,
    )


def _recover_scanned_pages(
    document, page_texts: list[str], scanned_pages: list[int]
) -> OcrOutcome:
    """Run OCR over the image-only pages, replacing their text in `page_texts`.

    `page_texts` is mutated in place: a page whose OCR result contains real text
    overwrites its (empty) text-layer entry. Pages OCR cannot read are left
    untouched, so the caller still gets whatever text the document did have.

    OCR failure on a single page is not fatal — a mostly-digital PDF with one
    bad scan should still upload. Only a completely unavailable Tesseract is
    reported up, and even then the caller decides whether that is an error.
    """
    if not scanned_pages:
        return OcrOutcome(attempted=False)

    available, reason = ocr_service.availability()
    if not available:
        return OcrOutcome(
            attempted=False,
            failed=list(scanned_pages),
            unavailable_reason=reason,
        )

    recovered: list[int] = []
    failed: list[int] = []

    for index, number in enumerate(scanned_pages):
        try:
            page_text = ocr_service.ocr_page(document[number - 1])
        except ocr_service.OcrUnavailableError as exc:
            # Tesseract gave out part-way through (removed, or missing language
            # pack). Stop rather than retrying every remaining page.
            logger.warning("OCR stopped at page %d: %s", number, exc)
            failed.extend(scanned_pages[index:])
            return OcrOutcome(True, recovered, failed, unavailable_reason=str(exc))
        except Exception as exc:  # pragma: no cover - unexpected rendering error
            logger.warning("OCR failed on page %d: %s", number, exc)
            failed.append(number)
            continue

        if has_text(page_text):
            page_texts[number - 1] = page_text.strip()
            recovered.append(number)
        else:
            failed.append(number)

    logger.info(
        "OCR recovered %d of %d image-only page(s)", len(recovered), len(scanned_pages)
    )
    return OcrOutcome(attempted=True, recovered=recovered, failed=failed)


def _pdf_method(text_layer_pages: int, outcome: OcrOutcome) -> str:
    """Describe how the text was obtained, for display in the UI.

    Keyed on whether any page had a real text layer, not on how many pages OCR
```

```
    recovered: a fully scanned PDF whose OCR partly failed is still an OCR
    extraction, and saying "PyMuPDF text layer" there would be a lie.
    """
    if not outcome.recovered:
        return "PyMuPDF text layer"
    if text_layer_pages == 0:
        return f"Tesseract OCR ({len(outcome.recovered)} page(s))"
    return f"PyMuPDF text layer + Tesseract OCR ({len(outcome.recovered)} page(s))"


def _pdf_warnings(
    page_count: int, scanned_pages: list[int], outcome: OcrOutcome
) -> list[str]:
    """Non-fatal notes the user should see about how the PDF was read."""
    warnings: list[str] = []

    if not scanned_pages:
        return warnings

    listed = ", ".join(str(n) for n in scanned_pages[:10])
    if len(scanned_pages) > 10:
        listed += ", ..."

    if outcome.attempted:
        if outcome.recovered:
            warnings.append(
                f"{len(outcome.recovered)} of {page_count} page(s) had no text "
                f"layer and were read with OCR: page {listed}. OCR output may "
                "contain recognition errors."
            )
        if outcome.failed:
            warnings.append(
                f"OCR produced no readable text for {len(outcome.failed)} page(s): "
                f"page {', '.join(str(n) for n in outcome.failed[:10])}. "
                "The summary will be based on the remaining pages."
            )
    else:
        warnings.append(
            f"{len(scanned_pages)} of {page_count} page(s) appear to be images and "
            f"were skipped because OCR is unavailable ({outcome.unavailable_reason}). "
            "Text from the remaining pages has been used. See the README for how "
            "to install Tesseract."
        )

    return warnings


def _no_text_detail(outcome: OcrOutcome) -> str:
    """Explain a PDF that yielded nothing, with the right remedy for the cause."""
    if outcome.unavailable_reason:
        return (
            "This PDF contains no text layer (every page is a scanned image) "
            f"and OCR could not run: {outcome.unavailable_reason}. Install "
            "Tesseract (see the README) or set TESSERACT_CMD in the backend .env, "
            "then upload the file again."
        )
    if outcome.attempted:
        return (
            "No readable text could be extracted. Every page is an image, and OCR "
            "did not recognise any text in them. The scan may be too low-quality "
            "or in a language that is not configured (OCR_LANGUAGE)."
        )
    return (
        "No text layer was found in this PDF, and no images were detected to run "
        "OCR on."
    )


def _extract_docx(path: Path) -> ExtractionResult:
    """Read paragraphs and tables from a DOCX file.
```

```
    Headers and footers are skipped on purpose: they repeat on every page, so
    including them would duplicate the same lines many times.
    """
    try:
        document = DocxDocument(str(path))
    except Exception as exc:
        raise HTTPException(
            status_code=422,
            detail=(
                "The DOCX file could not be read. It may be corrupt, or it may "
                f"be a legacy .doc file renamed to .docx. ({exc})"
            ),
        )

    blocks = [p.text.strip() for p in document.paragraphs if p.text.strip()]

    # Tables often hold the substance of a report (results, metrics).
    for table in document.tables:
        for row in table.rows:
            cells = [cell.text.strip() for cell in row.cells]
            if any(cells):
                blocks.append(" | ".join(cells))

    text = BLOCK_SEPARATOR.join(blocks)

    if not has_text(text):
        raise HTTPException(
            status_code=400,
            detail=(
                "No readable text was found in the DOCX file. If the content is "
                "inside images, it cannot be extracted without OCR."
            ),
        )

    return ExtractionResult(
        text=text,
        file_type="docx",
        extraction_method="python-docx (paragraphs and tables)",
        # python-docx has no layout engine, so page breaks are unknowable.
        page_count=None,
        ocr_used=False,
    )


def _extract_txt(path: Path) -> ExtractionResult:
    """Read a text file, tolerating the encodings Windows editors produce."""
    raw = path.read_bytes()

    # utf-8-sig also strips a byte-order mark. latin-1 maps every byte value, so
    # it always succeeds as the final fallback.
    text, encoding = "", "utf-8"
    for candidate in ("utf-8-sig", "cp1252", "latin-1"):
        try:
            text, encoding = raw.decode(candidate), candidate
            break
        except UnicodeDecodeError:
            continue

    text = text.replace("\r\n", "\n").replace("\r", "\n")

    if not has_text(text):
        raise HTTPException(status_code=400, detail="The text file is empty.")

    warnings = []
    if encoding not in ("utf-8-sig", "utf-8"):
        warnings.append(
            f"The file was not valid UTF-8 and was decoded as {encoding}; "
            "some characters may be misread."
        )

    return ExtractionResult(
```

```
        text=text,
        file_type="txt",
        extraction_method=f"plain text ({encoding})",
        page_count=None,
        ocr_used=False,
        warnings=warnings,
    )


# ---------------------------------------------------------------------------- #
# Storage
# ---------------------------------------------------------------------------- #
def _text_file(document_id: str) -> Path:
    return settings.upload_dir / f"{document_id}.txt"


def _meta_file(document_id: str) -> Path:
    return settings.upload_dir / f"{document_id}.json"


def build_metadata(
    document_id: str,
    filename: str,
    result: ExtractionResult,
    cleaned: CleanedText,
) -> dict:
    """Assemble the public metadata for an ingested document (no text).

    Counts describe the *cleaned* text, because that is what gets stored and
    summarized; the raw size is reported separately under `cleaning` so the
    effect of cleaning stays visible.
    """
    return {
        "document_id": document_id,
        "filename": filename,
        "file_type": result.file_type,
        "character_count": count_characters(cleaned.text),
        "word_count": count_words(cleaned.text),
        "chunk_count": len(chunking_service.chunk_text(cleaned.text)),
        "page_count": result.page_count,
        "ocr_used": result.ocr_used,
        "extraction_method": result.extraction_method,
        "cleaning": {
            "raw_character_count": cleaned.original_character_count,
            "removed_line_count": cleaned.removed_line_count,
            "hyphenation_joins": cleaned.hyphenation_joins,
        },
    }


def save_document(document_id: str, text: str, metadata: dict) -> None:
    """Persist the extracted text and its metadata."""
    settings.upload_dir.mkdir(parents=True, exist_ok=True)
    _text_file(document_id).write_text(text, encoding="utf-8")
    _meta_file(document_id).write_text(
        json.dumps(metadata, indent=2, ensure_ascii=False), encoding="utf-8"
    )


def load_document(document_id: str) -> tuple[str, dict]:
    """Return `(text, metadata)` for a stored document.

    Raises 404 if it is missing — which is also what the user sees after the
    server restarts with a cleared uploads directory.
    """
    meta_file, text_file = _meta_file(document_id), _text_file(document_id)

    if not meta_file.exists() or not text_file.exists():
        raise HTTPException(
            status_code=404,
            detail="Document not found or no longer available. Please upload it again.",
```

```
        )

    return text_file.read_text(encoding="utf-8"), json.loads(
        meta_file.read_text(encoding="utf-8")
    )


def document_exists(document_id: str) -> bool:
    return _meta_file(document_id).exists() and _text_file(document_id).exists()
```

backend/app/services/ocr_service.py

```
"""
Tesseract OCR fallback for PDF pages that carry no text layer.

Why this exists
---------------
A scanned PDF stores each page as a picture. PyMuPDF reads the text layer and
finds nothing, because there is no text layer — the characters exist only as
pixels. Tesseract recovers them from a rendered image of the page.

This module is called only for pages whose text layer came back (nearly) empty,
so a normal digital PDF never pays the cost of rendering and OCR: see
`document_service._extract_pdf`.

Tesseract is an external program, not a pip package
---------------------------------------------------
`pip install pytesseract` installs the *wrapper*; the `tesseract` executable
must be installed separately (see the root README). The two can disagree:
importing this module always succeeds, but OCR only works if the binary can be
found.

That is why nothing here raises at import time and why `availability()` exists:
callers ask first and can return a message telling the user how to fix it,
rather than failing with a 500 somewhere deep in the stack.
"""

import logging

import pytesseract
from PIL import Image

from app.config import settings

logger = logging.getLogger(__name__)


class OcrUnavailableError(RuntimeError):
    """The Tesseract binary could not be located, started or used."""


# Probing for the binary spawns a subprocess, so the answer is cached. `None`
# means "not checked yet" — deliberately not computed at import time, so that
# importing this module has no side effects and costs nothing.
_available: bool | None = None
_reason: str | None = None


def _configure() -> None:
    """Point pytesseract at a specific binary when one is configured.

    Needed when Tesseract was installed somewhere its installer did not add to
    PATH — which is the normal situation on Windows.
    """
    if settings.TESSERACT_CMD:
        pytesseract.pytesseract.tesseract_cmd = settings.TESSERACT_CMD
```

```
def availability() -> tuple[bool, str | None]:
    """Return `(available, reason)`; `reason` is `None` when OCR can run.

    The result is cached after the first call. Restarting the server (or
    installing Tesseract and calling `reset_cache()`) re-probes.
    """
    global _available, _reason
    if _available is not None:
        return _available, _reason

    _configure()
    try:
        version = pytesseract.get_tesseract_version()
        logger.info("Tesseract %s available for OCR", version)
        _available, _reason = True, None
    except pytesseract.TesseractNotFoundError:
        _available = False
        _reason = (
            f"the Tesseract program could not be started from "
            f"TESSERACT_CMD='{settings.TESSERACT_CMD}'"
            if settings.TESSERACT_CMD
            else "the Tesseract program is not installed or is not on PATH"
        )
    except Exception as exc:  # pragma: no cover - environment specific
        _available, _reason = False, f"the Tesseract check failed ({exc})"

    if not _available:
        logger.warning("OCR is unavailable: %s", _reason)

    return _available, _reason


def is_available() -> bool:
    """True if OCR can actually run right now."""
    return availability()[0]


def reset_cache() -> None:
    """Forget the cached probe. Exists so tests can re-check after a change."""
    global _available, _reason
    _available, _reason = None, None


def render_page(page) -> Image.Image:
    """Rasterise a PDF page for OCR at the configured DPI.

    `alpha=False` guarantees three bytes per pixel, so the buffer maps onto
    Pillow's "RGB" mode without an extra conversion. DPI matters: at the default
    72 DPI Tesseract misreads small text, which is why OCR_DPI defaults to 200.
    """
    pixmap = page.get_pixmap(dpi=settings.OCR_DPI, alpha=False)
    return Image.frombytes("RGB", (pixmap.width, pixmap.height), pixmap.samples)


def ocr_page(page) -> str:
    """Render one PDF page and return the text Tesseract reads from it.

    Raises `OcrUnavailableError` if Tesseract cannot be used at all. A page that
    simply contains no legible text yields an empty string, which is not an
    error — the caller decides what that means.
    """
    available, reason = availability()
    if not available:
        raise OcrUnavailableError(reason or "Tesseract is unavailable.")

    image = render_page(page)

    try:
        return pytesseract.image_to_string(image, lang=settings.OCR_LANGUAGE)
    except pytesseract.TesseractError as exc:
        # e.g. the requested language pack is not installed.
```

```
        raise OcrUnavailableError(
            f"Tesseract could not process the page (language "
            f"'{settings.OCR_LANGUAGE}'): {exc}"
        ) from exc
```

backend/app/services/cleaning_service.py

```
"""
Text cleaning for extracted documents.

Extraction (Phases 2-3) returns what the file literally contains, which is not
what a summarizer wants to read:

* a PDF page is a grid of lines, so every wrapped line ends in a newline that
  carries no meaning;
* a word broken across two lines keeps its hyphen ("summa-\\nrization");
* running headers, footers and page numbers repeat on every page;
* one character can arrive as several code points (ligatures, curly quotes,
  non-breaking spaces), so a ligature "fi" does not compare equal to a typed
  "fi".

This module turns that into flowing prose. It is deliberately conservative: a
line is only removed when it is structurally meaningless (a page number) or
provably repeated (a running header), because deleting real content is a far
worse failure than leaving a stray footer behind.

`clean_text` is the only entry point callers need. The steps it composes are
separate functions over a list of lines so each can be tested on its own.

Non-ASCII characters are written with `chr()` throughout. Several of the ones
handled here are invisible or indistinguishable from ASCII in source, and a
literal non-breaking space in this file would be a trap for the next editor.
"""

import logging
import re
import unicodedata
from collections import Counter
from dataclasses import dataclass

from app.config import settings

logger = logging.getLogger(__name__)

# Paragraphs are separated by a blank line, matching document_service's
# BLOCK_SEPARATOR, so cleaning neither invents nor destroys block boundaries.
PARAGRAPH_SEPARATOR = "\n\n"

# Characters that matter below, named once so the intent is readable.
NBSP = chr(0x00A0)
FIGURE_SPACE = chr(0x2007)
NARROW_NBSP = chr(0x202F)
EN_DASH = chr(0x2013)
EM_DASH = chr(0x2014)
SOFT_HYPHEN = chr(0x00AD)
MINUS_SIGN = chr(0x2212)
ELLIPSIS = chr(0x2026)


@dataclass
class CleanedText:
    """The cleaned text plus what changed, so the effect is visible, not silent."""

    text: str
    original_character_count: int
    original_line_count: int
    removed_line_count: int = 0
    hyphenation_joins: int = 0
```

```
    @property
    def character_count(self) -> int:
        return len(self.text)


# ---------------------------------------------------------------------------- #
# Step 1 - character normalisation
# ---------------------------------------------------------------------------- #
# NFKC folds the compatibility cases: ligatures (fi, fl, ffi), full-width forms
# and superscripts - and, less obviously, non-breaking spaces and the ellipsis,
# both of which have compatibility decompositions. The replacements below are
# the ones NFKC does *not* handle, chiefly curly quotes and en/em dashes.
#
# `str.maketrans` is required, not cosmetic: `str.translate` looks up ordinals,
# so a dict keyed by single-character strings is silently ignored - every entry
# here would be a no-op.
_CHARACTER_MAP = str.maketrans(
    {
        NBSP: " ",
        FIGURE_SPACE: " ",
        NARROW_NBSP: " ",
        chr(0x2018): "'",  # curly single quotes...
        chr(0x2019): "'",
        chr(0x201A): "'",
        chr(0x201B): "'",
        chr(0x201C): '"',  # ...and double quotes
        chr(0x201D): '"',
        chr(0x201E): '"',
        EN_DASH: "-",
        EM_DASH: "-",
        MINUS_SIGN: "-",
        ELLIPSIS: "...",
    }
)

# Removed outright: zero-width marks, word joiner and the byte-order mark.
# Invisible, but they split tokens and defeat n-gram matching.
_INVISIBLE = dict.fromkeys([0x200B, 0x200C, 0x200D, 0x2060, 0xFEFF], None)

# Keep tab and newline; drop the remaining C0 controls and DEL.
_CONTROL_CHARS = re.compile(r"[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]")


def normalise_characters(text: str) -> str:
    """Fold equivalent code points together so later matching is reliable.

    Quote and dash normalisation is not cosmetic. Evaluation compares generated
    summaries against a reference, and "don't" spelled with a curly apostrophe
    will not match "don't" spelled with a straight one.

    Soft hyphens survive this step on purpose: the de-hyphenation step needs to
    see them to know a word was broken.
    """
    text = unicodedata.normalize("NFKC", text)
    text = text.translate(_INVISIBLE)
    text = text.translate(_CHARACTER_MAP)
    return _CONTROL_CHARS.sub("", text)


# ---------------------------------------------------------------------------- #
# Step 2 - page numbers
# ---------------------------------------------------------------------------- #
# Bare digits are the dangerous case: "2024" alone on a line may be real content
# (a year, a table cell). A lone number is therefore only treated as pagination
# when several appear, which is what a run of pages looks like. Labelled forms
# ("Page 12") are unambiguous and always removed.
_DASHES = f"{EN_DASH}{EM_DASH}"
_PAGE_LABEL = re.compile(
    r"^\s*(?:page|pg\.?|p\.)\s*\d{1,4}(?:\s*(?:of|/)\s*\d{1,4})?\s*$",
    re.IGNORECASE,
```

```
)
_PAGE_OF_TOTAL = re.compile(r"^\s*\d{1,4}\s*(?:of|/)\s*\d{1,4}\s*$")
_PAGE_DECORATED = re.compile(
    rf"^\s*[-{_DASHES}]\s*\d{{1,4}}\s*[-{_DASHES}]\s*$"
)
_BARE_NUMBER = re.compile(r"^\s*\d{1,4}\s*$")

# Below this many bare-number lines they are assumed to be content, not pages.
MIN_BARE_NUMBERS = 3


def _is_page_number(line: str, bare_numbers: int) -> bool:
    if _PAGE_LABEL.match(line) or _PAGE_OF_TOTAL.match(line):
        return True
    if _PAGE_DECORATED.match(line):
        return True
    return bare_numbers >= MIN_BARE_NUMBERS and _BARE_NUMBER.match(line) is not None


def remove_page_number_lines(lines: list[str]) -> tuple[list[str], int]:
    """Drop lines that are a page number and nothing else."""
    bare_numbers = sum(1 for line in lines if _BARE_NUMBER.match(line))
    kept = [line for line in lines if not _is_page_number(line, bare_numbers)]
    return kept, len(lines) - len(kept)


# ---------------------------------------------------------------------------- #
# Step 3 - running headers and footers
# ---------------------------------------------------------------------------- #
def _comparable(line: str) -> str:
    """The form two lines are compared in: case- and spacing-insensitive."""
    return re.sub(r"\s+", " ", line).strip().casefold()


def remove_repeated_lines(
    lines: list[str], min_repeats: int, max_chars: int
) -> tuple[list[str], int]:
    """Drop short lines that recur throughout the document.

    A header or footer is the same text on every page, so exact repetition is
    strong evidence. Restricting to short lines keeps genuine content safe: a
    sentence that happens to repeat is still content, however often it occurs.

    Only *exact* repeats are caught. A footer whose text changes per page
    ("Report 2024 - page 3") is missed, unless the changing part is the whole
    line, in which case the page-number step above deals with it. That blind
    spot is deliberate - masking the digits would also delete genuinely
    numbered short lines such as a run of table captions.
    """
    comparable = [_comparable(line) for line in lines]
    counts = Counter(
        text for text in comparable if text and len(text) <= max_chars
    )
    repeated = {text for text, count in counts.items() if count >= min_repeats}

    kept = [
        line
        for line, text in zip(lines, comparable)
        if not (text in repeated and len(text) <= max_chars)
    ]
    return kept, len(lines) - len(kept)


# ---------------------------------------------------------------------------- #
# Step 4 - de-hyphenation
# ---------------------------------------------------------------------------- #
# A word broken across a line keeps its hyphen: "summa-\nrization". The join is
# only safe when the continuation starts lowercase, which is what distinguishes
# a broken word from a real compound ("State-\nOf-The-Art").
_BROKEN_WORD = re.compile(
    rf"([A-Za-z])[{re.escape(SOFT_HYPHEN)}-][ \t]*\n[ \t]*([a-z])"
```

```
)


def dehyphenate(text: str) -> tuple[str, int]:
    """Rejoin words split across a line break. Returns `(text, joins)`."""
    joins = 0

    def join(match: re.Match) -> str:
        nonlocal joins
        joins += 1
        return match.group(1) + match.group(2)

    text = _BROKEN_WORD.sub(join, text)

    # Any soft hyphen left was mid-word on a single line, where it is purely a
    # display hint and carries no characters of its own.
    return text.replace(SOFT_HYPHEN, ""), joins


# ---------------------------------------------------------------------------- #
# Step 5 - reflow
# ---------------------------------------------------------------------------- #
def reflow(text: str) -> str:
    """Turn a grid of lines back into paragraphs.

    Blank lines are honoured as paragraph breaks; a single newline is only a
    line wrap, so it becomes a space. Splitting each paragraph on whitespace
    also collapses runs of spaces and tabs, which is why no separate whitespace
    step is needed.
    """
    paragraphs = re.split(r"\n[ \t]*\n", text)
    joined = (" ".join(paragraph.split()) for paragraph in paragraphs)
    return PARAGRAPH_SEPARATOR.join(part for part in joined if part)


# ---------------------------------------------------------------------------- #
# Entry point
# ---------------------------------------------------------------------------- #
def clean_text(text: str) -> CleanedText:
    """Clean extracted text, keeping only structure that carries meaning.

    The order matters: de-hyphenation needs the original line breaks, so it runs
    before `reflow` removes them.
    """
    original_character_count = len(text)
    lines = normalise_characters(text).split("\n")
    original_line_count = len(lines)

    lines, page_number_lines = remove_page_number_lines(lines)

    header_lines = 0
    if settings.CLEAN_STRIP_HEADERS:
        lines, header_lines = remove_repeated_lines(
            lines, settings.HEADER_MIN_REPEATS, settings.HEADER_MAX_CHARS
        )

    reflowed, joins = dehyphenate("\n".join(lines))
    cleaned = reflow(reflowed)

    removed_line_count = page_number_lines + header_lines
    if removed_line_count or joins:
        logger.info(
            "Cleaning removed %d line(s) (%d page number, %d repeated) and "
            "rejoined %d hyphenated word(s)",
            removed_line_count,
            page_number_lines,
            header_lines,
            joins,
        )

    return CleanedText(
```

```
        text=cleaned,
        original_character_count=original_character_count,
        original_line_count=original_line_count,
        removed_line_count=removed_line_count,
        hyphenation_joins=joins,
    )
```

backend/app/services/chunking_service.py

```
"""
Sentence segmentation and sentence-aware chunking.

Chunking exists because of a hard model limit: BART reads 1024 tokens, so a
20-page document cannot be summarized in one pass. It must be split, summarized
piece by piece (map) and the results condensed (reduce). See the architecture
notes in the root README.

Two decisions shape this module:

* **Split on sentences, never mid-sentence.** A chunk that begins or ends in the
  middle of a sentence gives the summarizer a fragment, and fragments are what
  produce garbled output.
* **Overlap consecutive chunks.** Without it, a fact spanning a boundary is
  split across two chunks and may survive in neither. The overlap is measured in
  characters but applied on sentence boundaries, so it never cuts a sentence
  either.

Sentence segmentation uses NLTK's punkt model when its data is present, and a
regex splitter when it is not. Punkt is downloaded separately from the library
(`nltk.download('punkt_tab')`), exactly like the Tesseract binary, so a missing
model must degrade rather than crash. `sentence_tokenizer()` reports which one is
in use.
"""

import logging
import re
from dataclasses import dataclass

from app.config import settings

logger = logging.getLogger(__name__)

_tokenizer_name: str | None = None


# ---------------------------------------------------------------------------- #
# Sentence segmentation
# ---------------------------------------------------------------------------- #
def sentence_tokenizer() -> str:
    """Name of the tokenizer in use: `'nltk-punkt'` or `'regex'`."""
    global _tokenizer_name
    if _tokenizer_name is None:
        _tokenizer_name = _probe_tokenizer()
    return _tokenizer_name


def reset_tokenizer_cache() -> None:
    """Forget the probe result. Used by tests."""
    global _tokenizer_name
    _tokenizer_name = None


def _probe_tokenizer() -> str:
    """Decide once whether punkt can be used.

    The probe actually tokenizes a string rather than only checking that the
    data files exist, because NLTK 3.9 moved to `punkt_tab` and a machine can
    easily have the older `punkt` present but not the newer one.
```

```
    """
    try:
        from nltk.tokenize import sent_tokenize

        sent_tokenize("Probe. Two sentences.")
    except Exception as exc:  # LookupError, ImportError, ...
        logger.warning(
            "NLTK punkt data is unavailable (%s); using the regex sentence "
            "splitter. Run nltk.download('punkt_tab') for better segmentation.",
            exc,
        )
        return "regex"
    return "nltk-punkt"


def split_sentences(text: str) -> list[str]:
    """Split text into sentences, dropping empty results."""
    stripped = (text or "").strip()
    if not stripped:
        return []

    if sentence_tokenizer() == "nltk-punkt":
        from nltk.tokenize import sent_tokenize

        sentences = _repair_splits(sent_tokenize(stripped))
    else:
        sentences = _regex_split_sentences(stripped)

    return [sentence for sentence in (s.strip() for s in sentences) if sentence]


# Words that end in a full stop without ending a sentence.
_ABBREVIATIONS = frozenset(
    """
    mr mrs ms dr prof sr jr st vs etc inc ltd co corp gov sen rep gen col capt
    lt sgt fig eq ref no vol approx est al cf ed eds pp ph dist dept univ
    jan feb mar apr jun jul aug sep sept oct nov dec
    mon tue tues wed thu thur thurs fri sat sun
    """.split()
)

# Terminal punctuation, any closing quotes or brackets, then whitespace.
_BOUNDARY = re.compile(r"([.!?])([\"')\]]*)(\s+)")


def _is_false_boundary(text: str, match: re.Match) -> bool:
    """True if this full stop is part of an abbreviation, not a sentence end."""
    if match.group(1) != ".":
        return False  # "!" and "?" are unambiguous

    preceding = text[: match.start(1)]
    word = re.search(r"([A-Za-z]+|\d+)$", preceding)
    if word is None:
        return False

    token = word.group(1)
    if token.casefold() in _ABBREVIATIONS:
        return True

    if len(token) == 1:
        if token.isupper():
            return True  # an initial: "J. R. R. Tolkien"
        # A lone lowercase letter inside a dotted form: "e.g.", "i.e."
        if token.islower() and preceding[word.start() - 1 : word.start()] == ".":
            return True

    return False


def _regex_split_sentences(text: str) -> list[str]:
    """Fallback splitter for when punkt data is unavailable.
```

```
    Deliberately conservative: it breaks after terminal punctuation, except
    where the preceding token is a known abbreviation, an initial, or part of a
    dotted form. Decimals are safe automatically, because a full stop inside
    "3.14" is not followed by whitespace and so never matches the boundary.
    """
    sentences: list[str] = []
    start = 0

    for match in _BOUNDARY.finditer(text):
        if _is_false_boundary(text, match):
            continue
        sentence = text[start : match.end(2)].strip()
        if sentence:
            sentences.append(sentence)
        start = match.end(3)

    tail = text[start:].strip()
    if tail:
        sentences.append(tail)

    return sentences


# ------------------------------------------------------------------------ #
# Repairing punkt's splits
# ------------------------------------------------------------------------ #
# Punkt's English model is unsupervised, and its abbreviation set turns out to
# exclude some common ones: given "as shown in Fig. 3" it splits after "Fig.",
# and "reduce this risk, i.e. they select" after "i.e.". Those fragments are
# actively harmful downstream - an extractive summarizer can select
# "as shown in Fig." as a summary sentence - so the splits are rejoined using
# the same abbreviation knowledge the fallback splitter relies on.
_DOTTED_FORM = re.compile(r"[A-Za-z]\.[A-Za-z]\.")


def _ends_with_abbreviation(segment: str) -> bool:
    """True if this segment cannot really end where punkt ended it."""
    stripped = segment.rstrip().rstrip("\"')]")
    if not stripped.endswith("."):
        return False

    match = re.search(r"([A-Za-z.]+)$", stripped)
    if match is None:
        return False

    token = match.group(1)
    letters = token.replace(".", "")
    if not letters:
        return False

    if letters.casefold() in _ABBREVIATIONS:
        return True
    if _DOTTED_FORM.fullmatch(token):
        return True  # "e.g.", "i.e.", "a.m."
    return letters.isupper() and len(letters) == 1  # an initial: "J."


def _has_unclosed_bracket(segment: str) -> bool:
    """True if a bracket opened here was never closed."""
    return segment.count("(") > segment.count(")") or segment.count("[") > segment.count("]")


def _repair_splits(sentences: list[str]) -> list[str]:
    """Rejoin segments that punkt split at an abbreviation or mid-bracket.

    Repeated merging is intentional: "J. R. R. Tolkien" can arrive as several
    fragments, and each merge may reveal another abbreviation at the new end.
    """
    repaired: list[str] = []
    for sentence in sentences:
```

```
        if repaired and (
            _ends_with_abbreviation(repaired[-1]) or _has_unclosed_bracket(repaired[-1])
        ):
            repaired[-1] = f"{repaired[-1]} {sentence}"
        else:
            repaired.append(sentence)
    return repaired


# ------------------------------------------------------------------------ #
# Chunking
# ------------------------------------------------------------------------ #
@dataclass
class Chunk:
    """A slice of the document small enough to feed a model in one pass."""

    index: int
    text: str
    # Sentence segments joined into this chunk. A single sentence longer than
    # the chunk size is counted once per piece it was split into.
    sentence_count: int

    @property
    def character_count(self) -> int:
        return len(self.text)


def chunk_text(
    text: str, chunk_size: int | None = None, overlap: int | None = None
) -> list[Chunk]:
    """Split text into overlapping, sentence-aligned chunks.

    Every sentence in the input appears in at least one chunk. Overlap means
    text can appear in two chunks; nothing is ever dropped.
    """
    size = settings.CHUNK_SIZE if chunk_size is None else chunk_size
    budget = settings.CHUNK_OVERLAP if overlap is None else overlap

    if size <= 0:
        raise ValueError(f"chunk_size must be positive, got {size}")
    # The overlap must leave room for new content, or chunking could not
    # advance: a chunk that is all overlap carries nothing forward.
    budget = max(0, min(budget, size - 1))

    sentences = split_sentences(text)
    if not sentences:
        return []

    chunks: list[Chunk] = []
    current: list[str] = []

    for sentence in sentences:
        for piece in _split_oversized(sentence, size):
            if current and _joined_length(current) + 1 + len(piece) > size:
                chunks.append(_close(len(chunks), current))
                current = _overlap_tail(current, budget)
                # If the carried-over context leaves no room for this piece,
                # drop it: an overlap that blocks progress defeats its purpose.
                if _joined_length(current) + 1 + len(piece) > size:
                    current = []
            current.append(piece)

    if current:
        chunks.append(_close(len(chunks), current))

    logger.info(
        "Split %d sentence(s) into %d chunk(s) of up to %d characters",
        len(sentences),
        len(chunks),
        size,
    )
```

```
    return chunks


def _joined_length(sentences: list[str]) -> int:
    """Length of these sentences once joined, separators included."""
    return len(" ".join(sentences))


def _close(index: int, sentences: list[str]) -> Chunk:
    return Chunk(index=index, text=" ".join(sentences), sentence_count=len(sentences))


def _overlap_tail(sentences: list[str], budget: int) -> list[str]:
    """The trailing sentences to repeat at the start of the next chunk.

    Whole sentences only - carrying part of a sentence would reintroduce exactly
    the mid-sentence split this module exists to avoid.

    The final sentence is always carried when any overlap is requested, even if
    it alone exceeds `budget`. The purpose of an overlap is to bridge the
    boundary between two chunks, and a sentence that stops short of the boundary
    bridges nothing: skipping past it to shorter, older sentences duplicates
    text already wholly inside the previous chunk while leaving the seam bare.
    `budget` is therefore a target, not a hard cap. Older sentences are added
    only while they still fit.
    """
    if budget <= 0 or not sentences:
        return []

    taken = [sentences[-1]]
    total = len(sentences[-1])

    for sentence in reversed(sentences[:-1]):
        cost = len(sentence) + 1  # + the joining space
        if total + cost > budget:
            break
        taken.insert(0, sentence)
        total += cost

    return taken


def _split_oversized(sentence: str, limit: int) -> list[str]:
    """Break a sentence longer than `limit` on word boundaries.

    Rare, but a single unbroken "sentence" - a table row, a mangled OCR line
    with no punctuation - would otherwise produce a chunk no model can accept.
    """
    if len(sentence) <= limit:
        return [sentence]

    pieces: list[str] = []
    current = ""
    for word in sentence.split():
        if current and len(current) + 1 + len(word) > limit:
            pieces.append(current)
            current = word
        else:
            current = f"{current} {word}".strip()

    if current:
        pieces.append(current)
    return pieces or [sentence[:limit]]
```

backend/app/services/extractive_service.py

```
"""
Extractive summarization: pick the most representative sentences.
```

```
Approach (embeddings + TextRank):

1. Split the document into sentences.
2. Embed each one with all-MiniLM-L6-v2.
3. Build a graph whose nodes are sentences and whose edge weights are the
   cosine similarity between them.
4. Run PageRank over it and keep the highest-scoring sentences.

A sentence therefore scores highly when it is *similar to many others* - when it
is representative of the document - rather than when it is long or contains
frequent words. Nothing is rewritten: every returned sentence is quoted verbatim
from the source, which is what makes extractive summarization unable to
hallucinate.
"""

import logging
from functools import lru_cache

import networkx as nx
import numpy as np

from app.config import settings
from app.services.chunking_service import split_sentences

logger = logging.getLogger(__name__)

# Fraction of the document's sentences to keep for each requested length.
LENGTH_RATIOS = {"short": 0.15, "medium": 0.3, "long": 0.5}

# Below this many sentences there is nothing worth compressing.
MIN_SENTENCES = 3


@lru_cache(maxsize=1)
def _model():
    """The embedding model, loaded once per process and cached afterwards.

    Imported here rather than at module level so that starting the API does not
    pull in torch: only the first /api/summarize call pays for it.
    """
    from sentence_transformers import SentenceTransformer

    logger.info("Loading extractive model %s", settings.EXTRACTIVE_MODEL)
    return SentenceTransformer(settings.EXTRACTIVE_MODEL)


def rank_sentences(text: str) -> list[str]:
    """Rank sentences by TextRank score using MiniLM embeddings, returning sentences best-first."""
    sentences = split_sentences(text)
    if not sentences:
        return []
    if len(sentences) == 1:
        return sentences

    embeddings = _model().encode(sentences, normalize_embeddings=True)

    # Normalised embeddings make the dot product the cosine similarity.
    # Similarities are clipped at zero because PageRank needs non-negative
    # weights, and the diagonal is cleared so a sentence cannot reinforce itself.
    similarity = np.clip(embeddings @ embeddings.T, 0.0, None)
    np.fill_diagonal(similarity, 0.0)

    scores = nx.pagerank(nx.from_numpy_array(similarity))
    ranked_indices = sorted(scores, key=scores.get, reverse=True)
    return [sentences[i] for i in ranked_indices]


def summarize(text: str, length: str = "medium") -> str:
    """Return the most representative sentences, in their original order.
```

```
    Short documents are returned unchanged - compressing three sentences into
    one loses more than it gains.
    """
    sentences = split_sentences(text)
    if len(sentences) < MIN_SENTENCES:
        return text.strip()

    ranked = rank_sentences(text)
    keep = max(1, round(len(sentences) * LENGTH_RATIOS.get(length, 0.3)))

    top_k = ranked[:keep]
    top_k_counts = {}
    for s in top_k:
        top_k_counts[s] = top_k_counts.get(s, 0) + 1

    chosen = []
    for s in sentences:
        if top_k_counts.get(s, 0) > 0:
            chosen.append(s)
            top_k_counts[s] -= 1

    summary = " ".join(chosen)
    logger.info("Extractive: kept %d of %d sentence(s)", keep, len(sentences))
    return summary
```

backend/app/services/abstractive_service.py

```
"""
Abstractive summarization: generate new sentences that condense the source text.

Approach (BART + Map-Reduce):

1. Short documents (< 40 words) are returned unchanged.
2. Tokenize text using facebook/bart-large-cnn tokenizer.
3. If the input fits in a single chunk (<= 900 tokens), summarize directly.
4. Otherwise, use map-reduce:
   - Map phase: split document into overlapping sentence-aligned chunks using
     chunking_service. Summarize each chunk independently.
   - Reduce phase: concatenate partial summaries. If the result still exceeds 900
     tokens, chunk and summarize ONCE more (reduce) without looping indefinitely.

Abstractive models rewrite and synthesize concepts into fluent prose. However,
unlike extractive methods, they carry a slight risk of hallucination, which is why
a hybrid pipeline (Phase 7) is also provided.
"""

import logging
from functools import lru_cache

from app.config import settings
from app.services import chunking_service

logger = logging.getLogger(__name__)

# Target generation parameters per requested length.
LENGTH_SETTINGS = {
    "short": {"max_new_tokens": 120, "min_new_tokens": 40},
    "medium": {"max_new_tokens": 220, "min_new_tokens": 80},
    "long": {"max_new_tokens": 400, "min_new_tokens": 140},
}

MIN_WORDS = 40
MAX_SINGLE_PASS_TOKENS = 900
BART_MAX_INPUT_TOKENS = 1024


@lru_cache(maxsize=1)
def _model():
```

```
    """Load the BART tokenizer and sequence-to-sequence model once per process.

    Imported inside this function so starting the application or running non-abstractive
    tests does not trigger heavy PyTorch / transformers imports or model loading.
    """
    from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

    logger.info("Loading abstractive model %s on device %s", settings.ABSTRACTIVE_MODEL,
settings.ABSTRACTIVE_DEVICE)
    tokenizer = AutoTokenizer.from_pretrained(settings.ABSTRACTIVE_MODEL)
    model = AutoModelForSeq2SeqLM.from_pretrained(settings.ABSTRACTIVE_MODEL)
    model.to(settings.ABSTRACTIVE_DEVICE)
    model.eval()
    return tokenizer, model


def _generate_chunk_summary(text: str, length: str) -> str:
    """Summarize a single chunk of text that fits within the token limit."""
    tokenizer, model = _model()
    import torch

    params = LENGTH_SETTINGS.get(length, LENGTH_SETTINGS["medium"])
    inputs = tokenizer(
        text,
        truncation=True,
        max_length=BART_MAX_INPUT_TOKENS,
        return_tensors="pt",
    ).to(settings.ABSTRACTIVE_DEVICE)

    input_token_count = inputs["input_ids"].shape[1]
    min_tokens = min(params["min_new_tokens"], max(10, input_token_count - 5))
    max_tokens = max(min_tokens + 10, params["max_new_tokens"])

    with torch.no_grad():
        summary_ids = model.generate(
            **inputs,
            max_new_tokens=max_tokens,
            min_new_tokens=min_tokens,
            num_beams=4,
            early_stopping=True,
        )

    decoded = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
    return decoded.strip()


def summarize(text: str, length: str = "medium") -> str:
    """Generate an abstractive summary using BART with map-reduce chunking if needed."""
    stripped = text.strip()
    words = stripped.split()
    if len(words) < MIN_WORDS:
        return stripped

    tokenizer, _ = _model()
    token_ids = tokenizer.encode(stripped, truncation=False)

    # Fits in one chunk directly
    if len(token_ids) <= MAX_SINGLE_PASS_TOKENS:
        return _generate_chunk_summary(stripped, length)

    # Map step: chunk text and summarize each chunk
    chunks = chunking_service.chunk_text(stripped)
    if not chunks:
        return stripped

    logger.info("Abstractive map phase: summarizing %d chunks", len(chunks))
    partial_summaries = [_generate_chunk_summary(c.text, length) for c in chunks]
    joined_summary = " ".join(partial_summaries).strip()

    # Reduce step: check if joined partial summaries exceed token limit
    joined_tokens = tokenizer.encode(joined_summary, truncation=False)
```

```
    if len(joined_tokens) > MAX_SINGLE_PASS_TOKENS:
        logger.info("Abstractive reduce phase: joined summary (%d tokens) exceeds limit, reducing",
len(joined_tokens))
        reduce_chunks = chunking_service.chunk_text(joined_summary)
        if reduce_chunks:
            reduced_summaries = [_generate_chunk_summary(rc.text, length) for rc in reduce_chunks]
            return " ".join(reduced_summaries).strip()

    return joined_summary
```

backend/app/services/hybrid_service.py

```
"""
Hybrid summarization: combine extractive sentence selection with abstractive BART generation.

Approach (TextRank filter -> BART condensation):

1. Split document into sentences using chunking_service.
2. Short documents (< 3 sentences) are returned unchanged.
3. Extractive pre-filter: rank sentences with `extractive_service.rank_sentences(text)`
   (TextRank on sentence embeddings). Select the top ~50% of sentences by rank and
   restore their original document order.
4. Abstractive condensation: pass the joined top sentences to `abstractive_service.summarize(..., length)`
   to synthesize fluent prose.

By feeding BART only high-importance sentences, hybrid summarization reduces hallucination
risks while maintaining abstractive fluency and conciseness.
"""

import logging

from app.services import abstractive_service, chunking_service, extractive_service

logger = logging.getLogger(__name__)

MIN_SENTENCES = 3


def summarize(text: str, length: str = "medium") -> str:
    """Generate a hybrid summary by filtering sentences extractively then condensing abstractively."""
    sentences = chunking_service.split_sentences(text)
    if len(sentences) < MIN_SENTENCES:
        return text.strip()

    # Step 1: Rank sentences using TextRank
    ranked = extractive_service.rank_sentences(text)

    # Step 2: Select top ~50% of sentences by rank
    keep_count = max(1, round(len(sentences) * 0.5))
    top_half = ranked[:keep_count]

    top_counts = {}
    for s in top_half:
        top_counts[s] = top_counts.get(s, 0) + 1

    chosen = []
    for s in sentences:
        if top_counts.get(s, 0) > 0:
            chosen.append(s)
            top_counts[s] -= 1

    filtered_text = " ".join(chosen)
    logger.info(
        "Hybrid: filtered %d sentences down to top %d before abstractive pass",
        len(sentences),
        len(chosen),
    )
```

```
    # Step 3: Abstractive pass on the extractively pre-filtered text
    return abstractive_service.summarize(filtered_text, length=length)
```

backend/app/services/evaluation_service.py

```
"""
Evaluation service: evaluate generated summaries against reference summaries.

Metrics used:
- ROUGE-1: Unigram overlap between summary and reference (F1-score).
- ROUGE-2: Bigram overlap between summary and reference (F1-score).
- ROUGE-L: Longest Common Subsequence overlap (F1-score).
- BLEU: N-gram precision for sentence generation fluency (sacrebleu).

ROUGE scores are returned as plain Python floats in [0.0, 1.0].
BLEU score is returned as a plain Python float in [0.0, 100.0].
Numpy types from rouge_score are explicitly cast to float for JSON serialisation.
"""

import logging

import sacrebleu
from rouge_score import rouge_scorer

logger = logging.getLogger(__name__)


def evaluate(summary: str, reference: str) -> dict[str, float]:
    """Calculate ROUGE-1, ROUGE-2, ROUGE-L, and BLEU metrics for a summary against a reference text."""
    sum_text = summary.strip()
    ref_text = reference.strip()

    if not sum_text or not ref_text:
        return {
            "rouge_1": 0.0,
            "rouge_2": 0.0,
            "rouge_l": 0.0,
            "bleu": 0.0,
        }

    scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"], use_stemmer=True)
    rouge_results = scorer.score(ref_text, sum_text)

    bleu_result = sacrebleu.corpus_bleu([sum_text], [[ref_text]])

    return {
        "rouge_1": float(rouge_results["rouge1"].fmeasure),
        "rouge_2": float(rouge_results["rouge2"].fmeasure),
        "rouge_l": float(rouge_results["rougeL"].fmeasure),
        "bleu": float(bleu_result.score),
    }
```

frontend/src/services/api.js

```
/**
 * Thin wrapper around the FastAPI backend.
 *
 * Keeping every HTTP detail in this one module means:
 *   - components never build URLs or touch axios directly;
 *   - error shapes are normalised once, in one place, so the UI can always
 *     render `error.message` without defensive checks;
 *   - swapping axios for fetch (or changing the base URL) touches one file.
 */
import axios from 'axios'

/** Base URL of the FastAPI server. Override with VITE_API_BASE_URL in `.env`. */
```

```
export const API_BASE_URL =
  import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000'

const apiClient = axios.create({
  baseURL: API_BASE_URL,
  // Summarizing a long document on CPU is genuinely slow (map-reduce over many
  // chunks can take minutes), so the client timeout must be generous.
  timeout: 300_000,
})

/**
 * Collapse every possible failure mode into a single Error with a readable
 * `message` and an optional `status`.
 *
 * Handled cases: HTTP error responses (400/413/422/500), timeouts, the server
 * being unreachable, and anything unexpected.
 */
function normaliseError(error) {
  if (error.response) {
    const { status, data } = error.response
    let detail = data?.detail

    // FastAPI validation errors arrive as a list of objects; our backend
    // flattens them to a string, but stay defensive in case one slips through.
    if (Array.isArray(detail)) {
      detail = detail.map((item) => item?.msg ?? String(item)).join('; ')
    }
    if (typeof detail !== 'string' || !detail.trim()) {
      detail = `Request failed with status ${status}.`
    }

    const wrapped = new Error(detail)
    wrapped.status = status
    return wrapped
  }

  if (error.code === 'ECONNABORTED' || error.code === 'ETIMEDOUT') {
    return new Error(
      'The request timed out. Very long documents can take several minutes on CPU — try a shorter document
or the Extractive mode.',
    )
  }

  if (error.request) {
    return new Error(
      `Cannot reach the backend at ${API_BASE_URL}. Make sure the FastAPI server is running.`,
    )
  }

  return new Error(error.message || 'An unexpected error occurred.')
}

apiClient.interceptors.response.use(
  (response) => response,
  (error) => Promise.reject(normaliseError(error)),
)

/**
 * Check that the backend is alive and how it is configured.
 *
 * @returns {Promise<{status: string, service: string, version: string, python_version: string}>}
 */
export async function checkHealth() {
  const { data } = await apiClient.get('/api/health')
  return data
}

/**
 * Upload a document file (PDF, DOCX, or TXT).
 *
 * @param {File} file
```

```
 * @returns {Promise<UploadResponse>}
 */
export async function uploadDocument(file) {
  const formData = new FormData()
  formData.append('file', file)
  const { data } = await apiClient.post('/api/upload', formData, {
    headers: {
      'Content-Type': 'multipart/form-data',
    },
  })
  return data
}

/**
 * Request a summary for an uploaded document.
 *
 * @param {Object} params
 * @param {string} params.document_id
 * @param {string} [params.mode='extractive']
 * @param {string} [params.length='medium']
 * @returns {Promise<SummarizeResponse>}
 */
export async function summarizeDocument({ document_id, mode = 'extractive', length = 'medium' }) {
  const { data } = await apiClient.post('/api/summarize', {
    document_id,
    mode,
    length,
  })
  return data
}

/**
 * Request an evaluation of a summary against a reference text.
 *
 * @param {Object} params
 * @param {string} params.document_id
 * @param {string} params.reference
 * @param {string} [params.mode='extractive']
 * @param {string} [params.length='medium']
 * @returns {Promise<EvaluateResponse>}
 */
export async function evaluateDocument({ document_id, reference, mode = 'extractive', length = 'medium' })
{
  const { data } = await apiClient.post('/api/evaluate', {
    document_id,
    reference,
    mode,
    length,
  })
  return data
}

export default apiClient
```

5. Results / Output Screenshot

All planned development phases except containerised deployment were completed, and the system was verified end to end in the browser. This section records the functional results, compares the behaviour of the three summarization strategies, and shows the output screens of the application.

5.1 Functional Results

Capability	Result
PDF ingestion (digital)	Text layer extracted with PyMuPDF; page count and extraction method reported
PDF ingestion (scanned)	Image pages detected and routed to Tesseract; `ocr_used` true; per-page warnings reported
PDF ingestion (partly scanned)	Digital pages retained, image pages filled in; hybrid extraction method reported
DOCX ingestion	Paragraphs **and** tables extracted; page count reported as unknown rather than guessed
TXT ingestion	Encoding detected (UTF-8 / cp1252 / latin-1); a warning issued when not UTF-8
Text cleaning	Page numbers and repeated headers removed, hyphenated breaks repaired, prose reflowed; counts surfaced to the UI
Extractive summarization	Three length settings; output sentences verbatim; original document order preserved
Abstractive summarization	BART generation; single-pass for short documents, map-reduce for long ones
Hybrid summarization	Top-50% TextRank filtering followed by BART condensation
Evaluation	ROUGE-1, ROUGE-2, ROUGE-L (0.0–1.0) and BLEU (0–100) computed against a supplied reference
Error handling	Uniform JSON error shape; validation errors flattened; no stack traces leak to the client
Automated tests	89 tests passing

5.2 Behaviour of the Three Strategies

Observed behaviour matches the design intent, and the differences between the three modes are clearly visible on the same document.

Extractive. Output is a set of complete sentences lifted directly from the source. It reads as a collection of the document's load-bearing statements rather than as continuous prose, which is expected: sentences are selected for centrality, not for how they follow one another. Its indispensable property is that a generated sentence can be checked against the source by search — if it is in the summary, it is in the document, word for word.

Abstractive. Output reads as continuous, human-like prose and compresses more aggressively than the extractive mode. It is also the mode where the user should read critically: the model can state something

plausible and consistent with the surrounding text that the source does not in fact say.

Hybrid. Output retains the fluency of the abstractive mode, but inspection shows that it tracks the extractively selected sentences closely — it paraphrases central content rather than ranging over the whole document. This is the intended effect of constraining generation, and it is the mode that strikes the most useful balance for general reading.

5.3 Performance Observations

- **Model loading dominates the first request.** The first summarization call in a process pays for loading the selected model — roughly 90 MB for MiniLM and roughly 1.6 GB for BART. Subsequent calls reuse the cached instance. This is the reason for the `@lru_cache` singleton design.
- **Extractive mode is markedly faster than either generative mode**, since it performs one embedding pass plus a PageRank computation and generates no text.
- **Abstractive cost scales with document length**, roughly linearly in the number of chunks.
- **The event loop stays responsive during inference.** Because blocking work is dispatched to a worker thread, `/api/health` answers normally while a long document is being summarized.
- **Startup is cheap.** Because the heavy imports (`torch`, `transformers`) occur inside the cached getters rather than at module import, the API server starts in well under a second.
- **OCR cost is paid only where it is needed.** Pages with a text layer are never rendered, so a digital PDF pays nothing for the OCR capability.

5.4 Output Screenshots

The following figures show the application's screens. Each was captured from the running system (FastAPI backend on `http://localhost:8000`, React front end on `http://localhost:5173`).

Figure 5.1 — Upload screen. The landing screen with the backend-connection indicator and the file-upload control accepting PDF, DOCX and TXT files.

> *[Screenshot: Upload screen — Figure 5.1]*

Figure 5.2 — Document details and summarization controls. After a file is ingested, the extracted document's metadata (word and character count, chunk count, page count and extraction method) is shown alongside the mode selector (Extractive / Abstractive / Hybrid) and the length selector (Short / Medium / Long).

> *[Screenshot: Document details and controls — Figure 5.2]*

Figure 5.3 — Generated summary. The summary produced by the selected strategy, with the mode and length used to generate it.

> *[Screenshot: Generated summary — Figure 5.3]*

Figure 5.4 — ROUGE and BLEU evaluation. The evaluation panel, showing ROUGE-1, ROUGE-2, ROUGE-L and BLEU scores computed against a user-supplied reference summary.

> *[Screenshot: Evaluation panel — Figure 5.4]*

6. Conclusion

The AI Document Summarizer project stands as a strong example of how modern artificial intelligence and a well-structured system architecture can significantly improve the efficiency of reading and understanding long-form documents. This project set out to build a system that takes an arbitrary long-form document — possibly scanned, possibly poorly formatted, possibly far longer than any summarization model can read in one pass — and returns a concise summary, while giving the user control over the trade-off between factual fidelity and linguistic fluency.

That objective has been met. The completed system:

- ingests **PDF, DOCX and TXT** documents, extracting paragraphs and tables from DOCX, detecting text encodings in TXT and — for PDFs — routing pages with no text layer to **Tesseract OCR** while leaving digital pages untouched;
- **cleans** the extracted text into flowing prose, removing page numbers and repeated headers and repairing hyphenated line breaks, while deliberately erring toward keeping content;
- **chunks** documents on sentence boundaries with overlap, so that documents exceeding the model's 1024-token limit are summarized in full rather than silently truncated;
- implements **three summarization strategies** — extractive (embeddings + TextRank, structurally incapable of hallucination), abstractive (BART with map-reduce) and hybrid (TextRank filtering before BART condensation);
- **evaluates** any generated summary against a reference using ROUGE-1, ROUGE-2, ROUGE-L and BLEU;
- exposes all of this through a **typed REST API** and a responsive **single-page web interface**;
- and is validated by **89 automated tests**.

Several findings seem to us more generally useful than the application itself. First, **the pipeline matters as much as the model**: the quality of the summary depended at least as much on correct extraction, cleaning and chunking as on the choice of summarization model, and most of the engineering effort went into those stages. Second, **offering more than one strategy is not hedging** — the extractive, abstractive and hybrid modes fail in genuinely different ways, and which one is appropriate depends on the document and on the cost of being wrong. Third, **graceful degradation is a design decision, not an afterthought**: treating both Tesseract and the NLTK data as optional, and reporting their absence as data rather than as errors, is what makes the application installable and demonstrable on any machine rather than only on the machine it was developed on.

The work also has clearly understood limitations, and these point to the natural next steps: making the document store persistent so that summaries survive a restart; moving inference to a GPU or a quantised model to cut the time for long documents; adding multilingual and multi-document summarization; and packaging the system for containerised deployment. Together, these would turn a strong local demonstrator into a system suitable for wider practical use.

7. References

1. H. P. Luhn, "The Automatic Creation of Literature Abstracts," *IBM Journal of Research and Development*, vol. 2, no. 2, pp. 159–165, 1958.
2. R. Mihalcea and P. Tarau, "TextRank: Bringing Order into Texts," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2004, pp. 404–411.
3. L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford InfoLab Technical Report, 1999.
4. A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
5. M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in *Proceedings of ACL*, 2020.
6. C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 2004, pp. 74–81.
7. K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002, pp. 311–318.
8. I. Mani and M. T. Maybury (eds.), *Advances in Automatic Text Summarization*, MIT Press, 1999.
9. H. Edmundson, "New Methods in Automatic Extracting," *Journal of the ACM*, vol. 16, no. 2, pp. 264–285, 1969.
10. R. Smith, "An Overview of the Tesseract OCR Engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
11. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.
12. T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proceedings of EMNLP: System Demonstrations*, 2020, pp. 38–45.
13. FastAPI documentation — https://fastapi.tiangolo.com
14. Pydantic documentation — https://docs.pydantic.dev
15. React documentation — https://react.dev
16. Vite documentation — https://vite.dev
17. PyMuPDF documentation — https://pymupdf.readthedocs.io
18. Tesseract OCR (UB Mannheim Windows builds) — https://github.com/UB-Mannheim/tesseract/wiki
19. `rouge-score` — https://github.com/google-research/google-research/tree/master/rouge

20. `sacrebleu` — https://github.com/mjpost/sacrebleu

21. NetworkX documentation — https://networkx.org/documentation/stable/

22. NLTK documentation — https://www.nltk.org

MAINTAINED BY

Vinay Guleria · Anish Sharma · Rishav Thakur

PROJECT GUIDE

[Guide Name]

University Institute of Computing, Chandigarh University